

3.1 动态规划的设计思想

□ 采用动态规划求解的问题的一般要具有3个性质

- **性质1—最优性原理：** 如果问题的最优解所包含的子问题的解也是最优的，就称该问题具有最优子结构，即满足最优性原理。



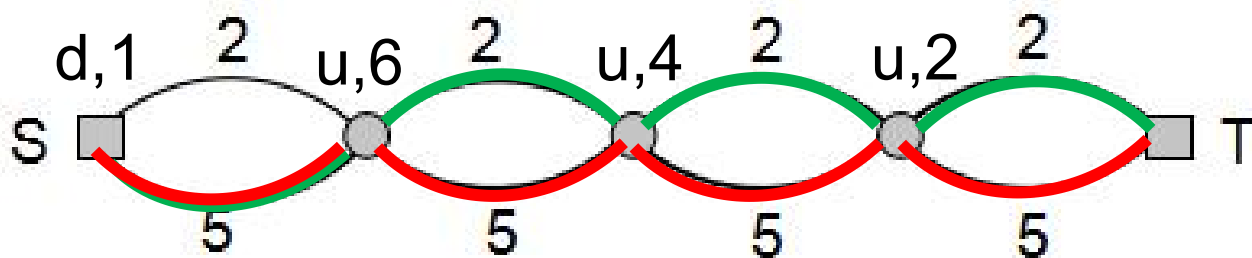
- **使用动态规划策略的条件：** 优化原则
- 优化原则：一个最优决策序列的任何子序列，一定是相对于子序列的初始和结束状态的最优决策序列。



3.1 动态规划的设计思想

➤ 不符合最优子结构性质 (优化原则) 的例子

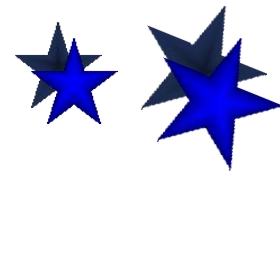
例：求总长模10的最小路径



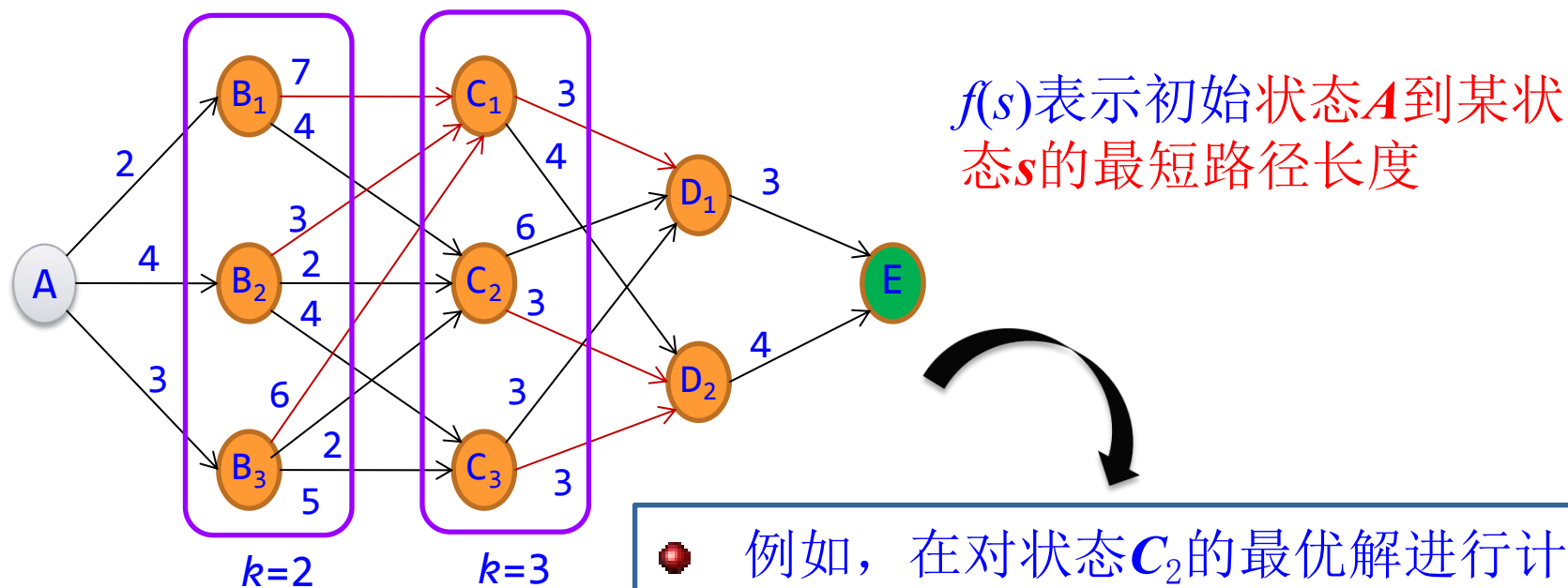
动态规划算法的解：下, 上, 上, 上

最优解：下, 下, 下, 下

该问题不满足优化原则，不能用动态规划。



- **性质2——无后效性：**即某阶段中的某状态对应的最优解一旦确定，该状态以后的决策过程不会影响该状态，只与当前状态有关。



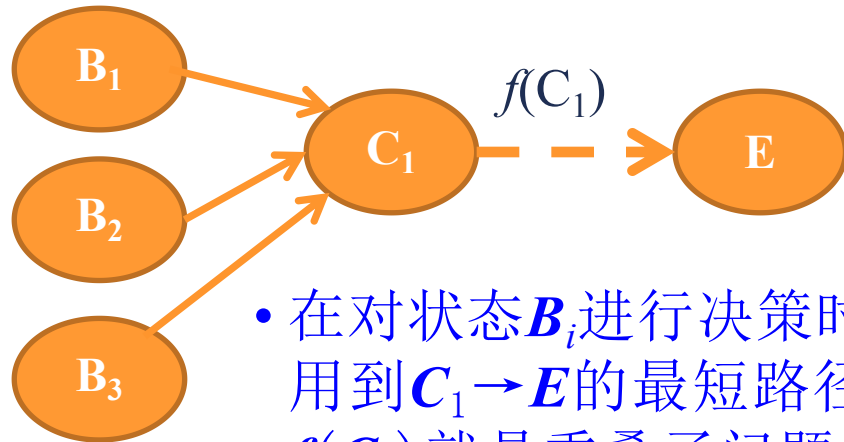
- 例如，在对状态 C_2 的最优解进行计算时，状态A, B_1 , B_2 , B_3 的最优解已经确定了。因此，在对 C_2 进行计算时，不会修改已经确定好的最优解。

动态规划求解的基本步骤

- 性质3——有重叠子问题：即子问题之间是不独立的，一个子问题在下一阶段决策中可能被多次使用到。（该性质并不是动态规划的必要条件，但是如果没有这条性质，动态规划算法同其他算法相比就不具备优势）。

求斐波那契数列 $f(6)$ 时：

- 先求 $f(5)$ ，后求 $f(4)$
- 求 $f(5)$ 时，也要求 $f(4)$
- $f(4)$ 就是公共子问题
- 可以在某阶段求出 $f(4)$ 后保存到 dp 表中，以后再次使用时，查表即可。



- 在对状态 B_i 进行决策时，都要用到 $C_1 \rightarrow E$ 的最短路径，因此 $f(C_1)$ 就是重叠子问题。
- 子问题的解存于数组 dp 中，再次使用时查表即可。

动态规划求解的基本步骤

● 动态规划方案设计有一定的模式，一般经历以下几个步骤

(1) 划分阶段：按照问题的时间或空间特征将问题划分成若干阶段。划分的阶段一定是有序的或可排序的，否则问题无法求解。

(2) 确定状态和状态变量：问题发展到不同阶段所对应的情况，用不同的状态变量表示，同一阶段的不同状态看成一个状态集合。

(3) 确定决策并写出状态转移方程：根据问题所要求解的目标（决策）给出从上一个阶段到本阶段的状态转移方程。

(4) 寻找边界条件：因为状态转移方程是一个递推式，因此需要一个终止条件（边界条件）。



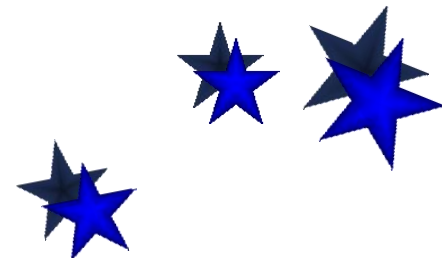
3.1 动态规划的设计思想

● 小结

(1) 动态规划(Dynamic Programming): 求解过程是多阶段决策过程, 每步处理一个子问题, 可用于求解组合优化问题。

(2) 求解步骤: 确定子问题的边界、从最小的子问题开始进行多步判断。

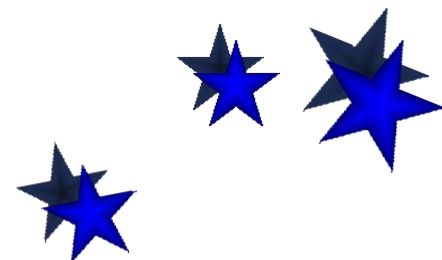
(3) 适用条件: 问题要满足优化原则或最优子结构性质, 即: 一个最优决策序列的任何子序列一定是相对于子序列的初始和结束状态的最优决策序列。



3.2 动态规划算法设计

■ 动态规划算法的设计要素

1. 问题建模，优化的目标函数是什么？约束条件是什么？
2. 如何划分子问题（边界）？
3. 问题的优化函数值与子问题的优化函数值存在着什么依赖关系？(递推方程)
4. 是否满足优化原则？
5. 最小子问题怎样界定？其优化函数值，即初值等于什么？



3.2 动态规划算法设计

■ 举例：矩阵链相乘

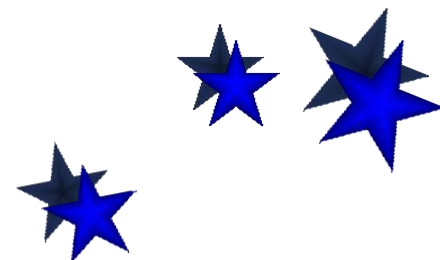
问题： 设 $A_1, A_2, \dots, A_n$ 为矩阵序列， A_i 为 $P_{i-1} \times P_i$ 阶矩阵， $i = 1, 2, \dots, n$. 试确定矩阵的乘法顺序，使得元素相乘的总次数最少.

输入： 向量

$$P = \langle P_0, P_1, \dots, P_n \rangle,$$

其中 $P_0, P_1, \dots, P_n$ 为 n 个矩阵的行数与列数

输出： 矩阵链乘法加括号的位置.



3.2 动态规划算法设计

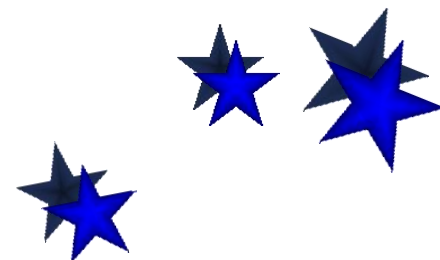
➤ 矩阵相乘基本运算次数

矩阵 A : i 行 j 列, B : j 行 k 列, 以元素相乘作基本运算, 计算 AB 的工作量

$$\begin{bmatrix} \dots \\ a_{t1} & a_{t2} & \dots & a_{tj} \\ \dots \end{bmatrix} \begin{bmatrix} b_{1s} \\ b_{2s} \\ \vdots \\ b_{js} \end{bmatrix} = \begin{bmatrix} c_{ts} \end{bmatrix}$$

$$c_{ts} = a_{t1} b_{1s} + a_{t2} b_{2s} + \dots + a_{tj} b_{js}$$

AB : i 行 k 列, 计算每个元素需要做 j 次乘法, 总计乘法次数为 ijk



3.2 动态规划算法设计

实例

实例: $P = \langle 10, 100, 5, 50 \rangle$

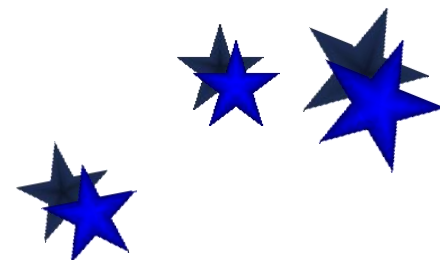
$A_1: 10 \times 100, A_2: 100 \times 5, A_3: 5 \times 50,$

乘法次序

$$\underline{(A_1 A_2)} A_3: 10 \times 100 \times 5 + 10 \times 5 \times 50 = \boxed{7500}$$

$$A_1 (A_2 A_3): 10 \times 100 \times 50 + 100 \times 5 \times 50 = 75000$$

第一种次序计算次数（乘法）最少



3.2 动态规划算法设计

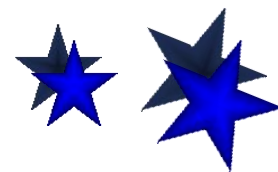
➤ 蛮力算法的时间复杂度分析

加 n 个括号的方法有 $\frac{1}{n+1} \binom{2n}{n}$ 种，
是一个Catalan数，是指数级别

$$W(n) = \Omega\left(\frac{1}{n+1} \frac{(2n)!}{n!n!}\right)$$

代入
Stirling
公式

$$= \Omega\left(\frac{1}{n+1} \frac{\sqrt{2\pi 2n} \left(\frac{2n}{e}\right)^{2n}}{\sqrt{2\pi n} \left(\frac{n}{e}\right)^n \sqrt{2\pi n} \left(\frac{n}{e}\right)^n}\right) = \Omega(2^{2n} / n^{\frac{3}{2}})$$



3.2 动态规划算法设计

➤ 动态规划算法实现

- 子问题划分

$A_{i..j}$: 矩阵链 $A_i A_{i+1} \dots A_j$, 边界 i, j

输入向量: $\langle P_{i-1}, P_i, \dots, P_j \rangle$

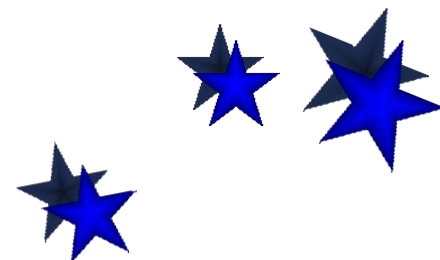
其最好划分的运算次数: $m[i, j]$

- 子问题的依赖关系

最优划分最后一次相乘发生在矩阵 k 的位置, 即

$$A_{i..j} = A_{i..k} A_{k+1..j}$$

$A_{i..j}$ 最优运算次数依赖于 $A_{i..k}$ 与 $A_{k+1..j}$ 的最优运算次数



3.2 动态规划算法设计

➤ 优化函数的递推方程

递推方程:

$m[i,j]$: 得到 $A_{i..j}$ 的最少的相乘次数

$m[i,j]$

$$= \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{ \underline{m[i,k]} + \underline{m[k+1,j]} + P_{i-1}P_kP_j \} & i < j \end{cases}$$

A_i	$\dots$	A_k	A_{k+1}	$\dots$	A_j
$P_{i-1} \times P_k$			$P_k \times P_j$		

该问题满足优化原则

设立标记函数: 为了确定加括号的次序, 设计表 $s[i,j]$, 记录求得最优 $m[i,j]$ 时的划分位置 k 。



3.2 动态规划算法设计

■ 动态规划算法的递归实现

➤ 矩阵相乘的递归算法

算法1 RecurMatrixChain (P, i, j)

子问
题 $i-j$

1. $m[i,j] \leftarrow \infty$

2. $s[i,j] \leftarrow i$

3. for $k \leftarrow i$ to $j-1$ do

划分
位置 k

4. $q \leftarrow$ RecurMatrixChain(P, i, k)
+ RecurMatrixChain($P, k+1, j$) + $p_{i-1}p_kp_j$

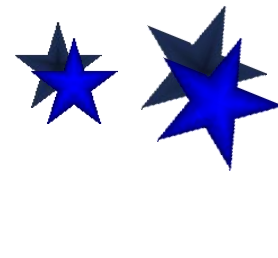
5. if $q < m[i,j]$

6. then $m[i,j] \leftarrow q$

7. $s[i,j] \leftarrow k$

8. return $m[i,j]$

找到更
好的解



3.2 动态规划算法设计

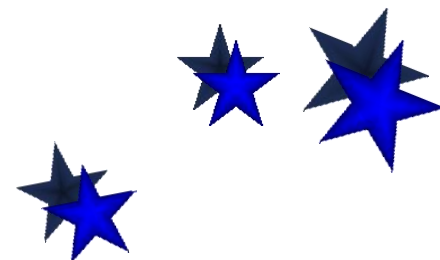
➤ 算法分析

- 时间复杂度的递推方程

$$T(n) \geq \begin{cases} O(1) & n = 1 \\ \sum_{k=1}^{n-1} (T(k) + T(n-k) + O(1)) & n > 1 \end{cases}$$

$$T(n) \geq O(n) + \sum_{k=1}^{n-1} T(k) + \sum_{k=1}^{n-1} T(n-k)$$

$$T(n) \geq O(n) + 2 \sum_{k=1}^{n-1} T(k)$$



3.2 动态规划算法设计

- 时间复杂度: $T(n) \geq 2^{n-1}$

数学归纳法证明:

$n=2$, 显然为真.

假设对于任何小于 n 的 k , 命题为真

证明 $k=n$ 时命题成立:

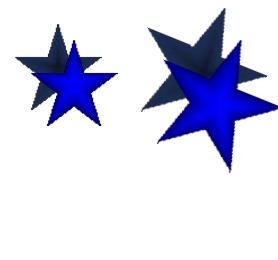
$$T(n) \geq O(n) + 2 \sum_{k=1}^{n-1} T(k)$$

代入归
纳假设

$$\geq O(n) + 2 \sum_{k=1}^{n-1} 2^{k-1}$$

等比数
列求和

$$= O(n) + 2(2^{n-1} - 1) \geq 2^{n-1}$$



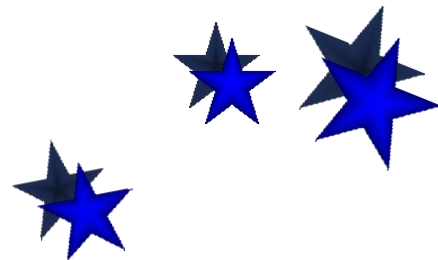
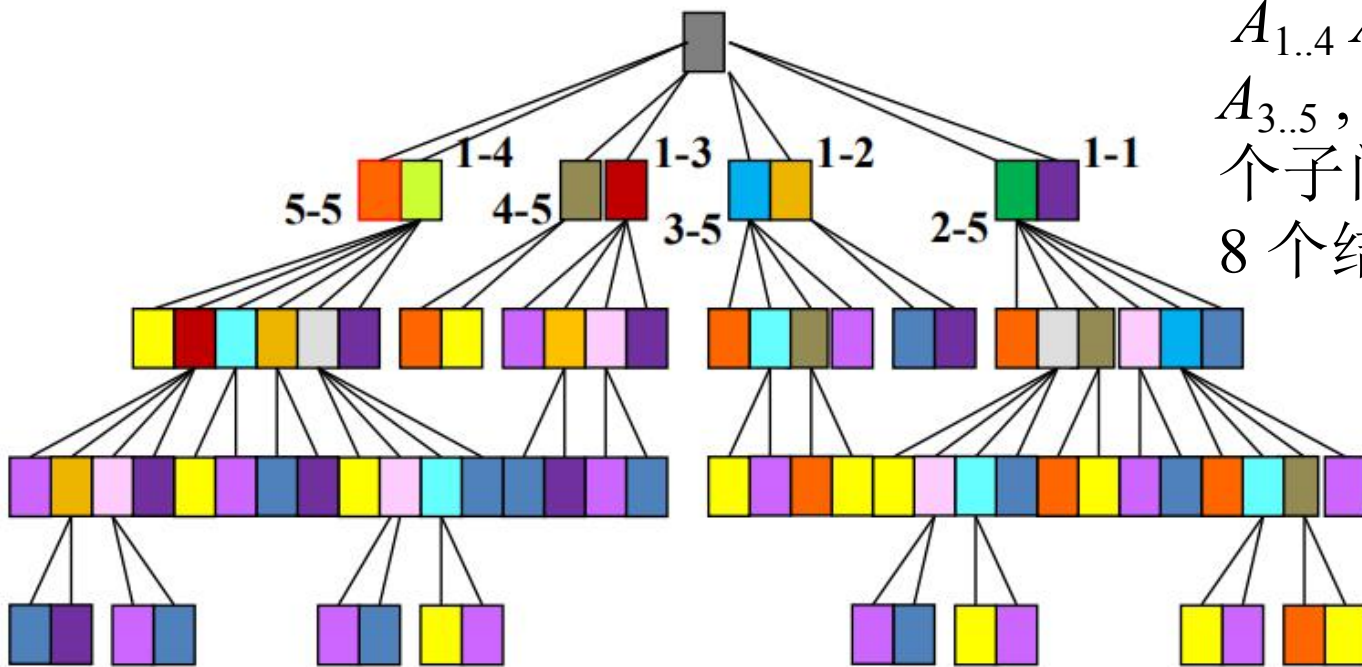
3.2 动态规划算法设计

➤ 递归实现时间复杂度高的原因分析

子问题的产生, $n=5$

划分:

$A_{1..4} A_{5..5}$, $A_{1..3} A_{4..5}$, $A_{1..2} A_{3..5}$, $A_{1..1} A_{2..5}$, 产生 8 个子问题, 即第一层的 8 个结点.



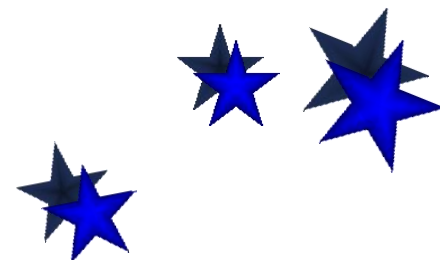
3.2 动态规划算法设计

子问题	1-1	2-2	3-3	4-4	5-5	1-2	2-3	3-4	4-5	1-3	2-4	3-5	1-4	2-5	1-5
数	8	12	14	12	8	4	5	5	4	2	2	2	1	1	1

$n=5$, 不同的子问题: 15个, 计算子问题: 81个。

复杂度高的原因: 相同的子问题需要重复计算

解决方案: 使用恰当的数据结构存储子问题的结果, 以避免重复计算。



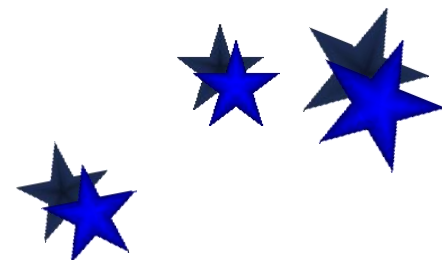
3.2 动态规划算法设计

- 动态规划递归实现小结

(1) 与蛮力算法相比较，动态规划算法利用了子问题优化函数间的依赖关系（递推公式），时间复杂度有所降低。

(2) 动态规划算法的递归实现效率不高，原因在于同一子问题多次重复出现，每次出现都需要重新计算一遍。

(3) 采用空间换时间策略，记录每个子问题首次计算结果（记录备忘录），后面再用时就直接取值，每个子问题只算一次。

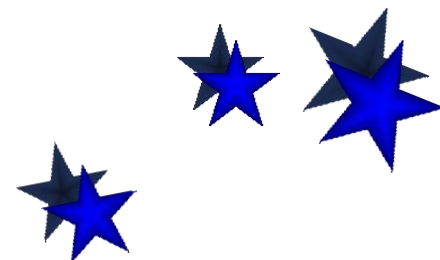


3.2 动态规划算法设计

■ 动态规划算法的迭代实现

➤ 迭代计算的关键

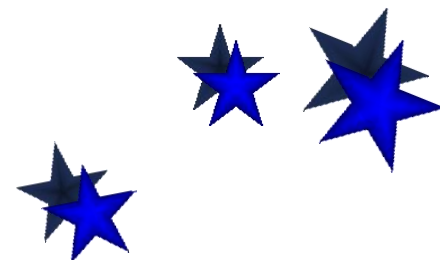
- 每个子问题只计算一次
- 迭代过程
 - 从最小的子问题算起
 - 考虑计算顺序，以保证后面用到的值前面已经计算好
 - 存储结构保存计算结果——备忘录
- 解的追踪
 - 设计标记函数标记每步的决策
 - 考虑根据标记函数追踪解的算法



3.2 动态规划算法设计

➤ 矩阵链乘法不同规模子问题个数分析

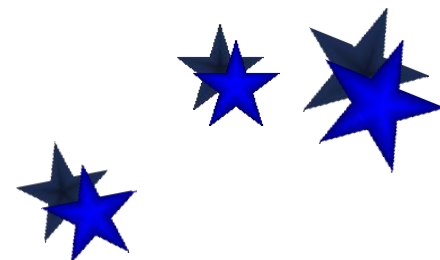
- 长度1: 只含1个矩阵, 有 n 个子问题(不需要计算)
- 长度2: 含2个矩阵, $n-1$ 个子问题
- 长度3: 含3个矩阵, $n-2$ 个子问题
- ...
- 长度 $n-1$: 含 $n-1$ 个矩阵, 2个子问题
- 长度 n : 原始问题, 只有1个



3.2 动态规划算法设计

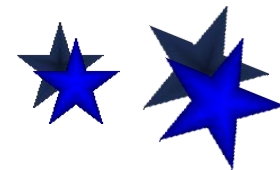
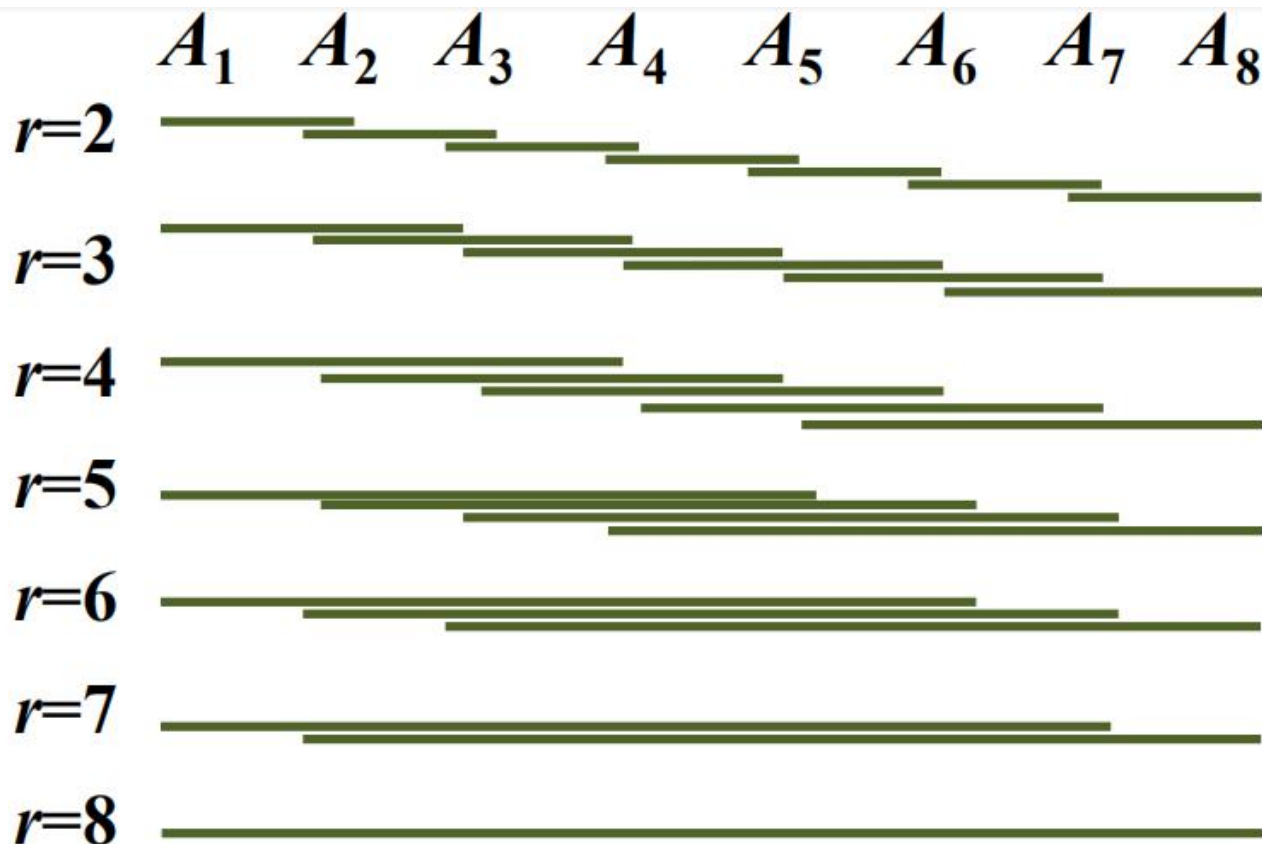
➤ 矩阵链乘法迭代顺序

- 长度为1: 初值, $m[i, i] = 0$ ——边界
- 长度为2: 1..2, 2..3, 3..4, ..., $n-1..n$
- 长度为3: 1..3, 2..4, 3..5, ..., $n-2..n$
- ...
- 长度为 $n-1$: 1.. $n-1$, 2.. n
- 长度为 n : 1.. n



3.2 动态规划算法设计

➤ $n=8$ 的子问题计算顺序



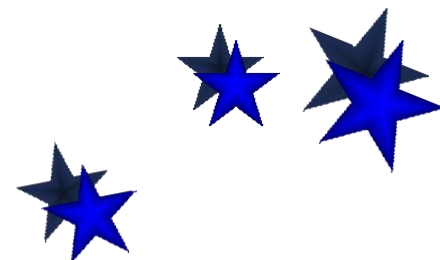
3.2 动态规划算法设计

➤ 实例

输入： $P = \langle 30, 35, 15, 5, 10, 20 \rangle$,
 $n = 5$

矩阵链： $A_1 A_2 A_3 A_4 A_5$ ，其中
 $A_1: 30 \times 35$, $A_2: 35 \times 15$, $A_3: 15 \times 5$,
 $A_4: 5 \times 10$, $A_5: 10 \times 20$

备忘录： 存储所有子问题的最小乘法次数及得到这个值的划分位置。

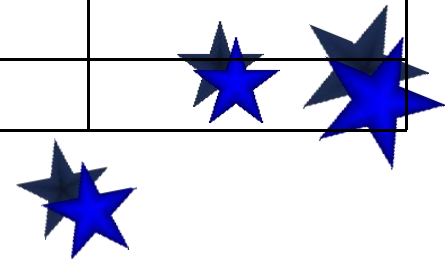


3.2 动态规划算法设计

备忘录：矩阵 m 标记函数： s

$r=1$	$m[1,1] = 0$	$m[2,2] = 0$	$m[3,3] = 0$	$m[4,4] = 0$	$m[5,5] = 0$
$r=2$	$m[1,2] = 15750$	$m[2,3] = 2625$	$m[3,4] = 750$	$m[4,5] = 1000$	
$r=3$	$m[1,3] = 7875$	$m[2,4] = 4375$	$m[3,5] = 2500$		
$r=4$	$m[1,4] = 9375$	$m[2,5] = 7125$			
$r=5$	$m[1,5] = 11875$				

$r=2$	$s[1,2] = 1$	$s[2,3] = 2$	$s[3,4] = 3$	$s[4,5] = 4$	
$r=3$	$s[1,3] = 1$	$s[2,4] = 3$	$s[3,5] = 3$		
$r=4$	$s[1,4] = 3$	$s[2,5] = 3$			
$r=5$	$s[1,5] = 3$				



3.2 动态规划算法设计

➤ 例如： $m[2,5]$ 代表矩阵： $A_2A_3A_4A_5$ 乘积的最少次数，括号的添加有如下可能：

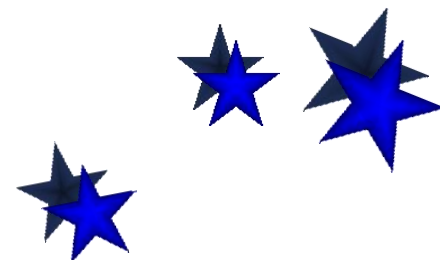
$$(1) \quad A_2 (A_3A_4A_5) = 0 + m[3,5] + 35 \times 15 \times 20 = 13000$$

$$(2) \quad (A_2A_3)(A_4A_5) = m[2,3] + m[4,5] + 35 \times 5 \times 20 = 7125$$

$$(3) \quad (A_2A_3A_4)A_5 = m[2,4] + 0 + 35 \times 10 \times 20 = 11375$$

$$(4) \quad m[2,5] = \min\{13000, 7125, 11375\} = 7125$$

$$\times s[2,5] = 3$$



3.2 动态规划算法设计

➤ 解的追踪:

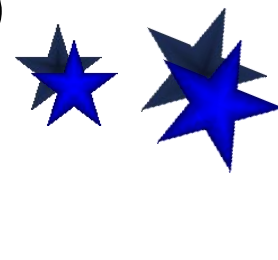
(1) 最少的乘法次数: $m[1,5]=11875$

(2) 加括号的位置

解的追踪: $s[1,5]=3 \Rightarrow (A_1A_2A_3)(A_4A_5)$

$s[1,3]=1 \Rightarrow A_1(A_2A_3)$

$A_1A_2A_3A_4A_5$ 的计算顺序为: $(A_1(A_2A_3))(A_4A_5)$

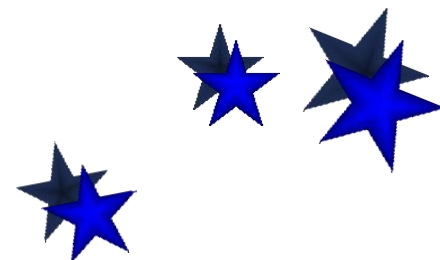


3.2 动态规划算法设计

➤ 迭代实现: **MatrixChain(P, n)**

1. 令所有的 $m[i, i]$ 初值为0
2. for $r \leftarrow 2$ to n do // r 为链长
3. for $i \leftarrow 1$ to $n - r + 1$ do // 左边界 i
4. $j \leftarrow i + r - 1$ // 右边界 j
5. $m[i, j] \leftarrow m[i + 1, j] + p_{i-1} p_i p_j$ // $k = i$
6. $s[i, j] \leftarrow i$ // 记录 k
7. for $k \leftarrow i + 1$ to $j - 1$ do // 遍历 k
8. $t \leftarrow \underline{m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j}$
9. if $t < m[i, j]$
10. then $m[i, j] \leftarrow t$ // 更新解
11. $s[i, j] \leftarrow k$

二维数组 m
与 s 为备忘录



3.2 动态规划算法设计

➤ 时间复杂度分析

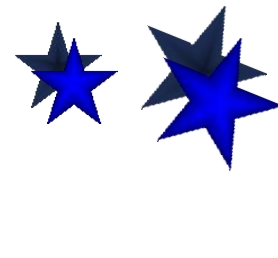
- **根据伪码**：行 2, 3, 7 都是 $O(n)$ ，循环执行 $O(n^3)$ 次，内部为 $O(1)$

$$W(n) = O(n^3)$$

- **根据备忘录**：估计每项工作量，求和。子问题有 $O(n^2)$ 个，确定每个子问题的最少乘法次数需要对不同划分位置比较，需要 $O(n)$ 时间。

$$W(n) = O(n^3)$$

- 追踪解工作量 $O(n)$ ，总工作量 $O(n^3)$ 。



3.2 动态规划算法设计

➤ 两种实现的比较

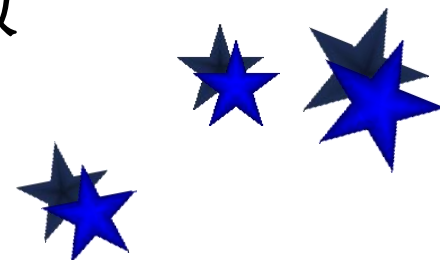
递归实现：时间复杂性高，空间较小

迭代实现：时间复杂性低，空间消耗多

原因：递归实现子问题多次重复计算，子问题计算次数呈指数增长. 迭代实现每个子问题只计算一次.

动态规划时间复杂度：

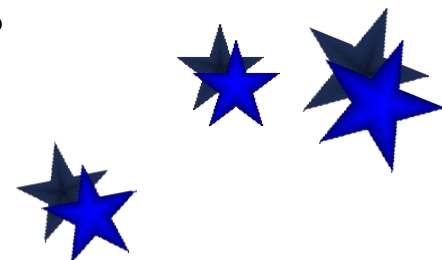
备忘录各项计算量之和 + 追踪解工作量. 通常追踪工作量不超过计算工作量，是问题规模的多项式函数



3.2 动态规划算法设计

➤ 动态规划算法的要素：

- 划分子问题，确定子问题边界，将问题求解转 变成多步判断的过程.
- 定义优化函数,以该函数极大(或极小) 值作为依据, 确定是否满足优化原则.
- 列优化函数的递推方程和边界条件
- 自底向上计算，设计备忘录 (表格)， 考虑是否需要设立标记函数（追踪解的数据结构）。
- 用递推方程或备忘录估计时间复杂度



3.3 动态规划算法的典型应用

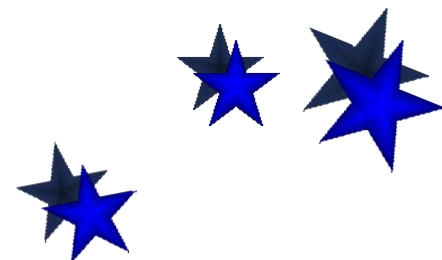
■ **投资问题**——资源分配问题：是将数量一定的一种或若干种资源（原材料、资金、设备或劳动力等），合理地分配给若干使用者，使总收益最大。

◆ 投资问题的建模

问题： m 元钱， n 项投资， $f_i(x)$ ：将 x 元投入第 i 个项目的效益。求使得总效益最大的投资方案。

建模：

问题的解是向量 $\langle x_1, x_2, \dots, x_n \rangle$ ，
 x_i 是投给项目 i 的钱数， $i=1, 2, \dots, n$.



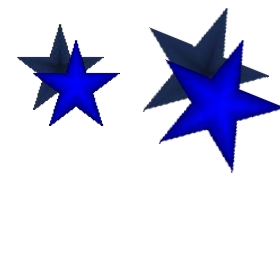
3.3 动态规划算法的典型应用

目标函数 $\max\{f_1(x_1)+f_2(x_2)+\dots+f_n(x_n)\}$

约束条件 $x_1+x_2+\dots+x_n=m, x_i \in \mathbb{N}$

◆ 实例： 5万元钱， 4个项目， 效益函数如下表所示：

x	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	0	0
1	11	0	2	20
2	12	5	10	21
3	13	10	30	22
4	14	15	32	23
5	15	20	40	24



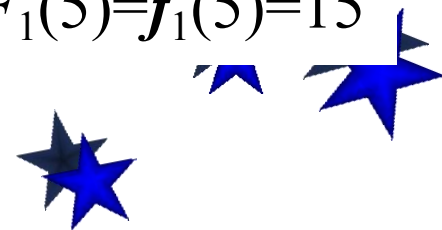
3.3 动态规划算法的典型应用

➤ 实例分析——分阶段决策

(1) 第1阶段：分别计算将 $x(0 \leq x \leq 5)$ 万元分配给前1个项目的最大收益

- 分配0万元给前1个项目的最大收益为0，即 $F_1(0)=f_1(0)=0$
- 分配1万元给前1个项目的最大收益为11，即 $F_1(1)=f_1(1)=11$
- 分配2万元给前1个项目的最大收益为12，即 $F_1(2)=f_1(2)=12$
- 分配3万元给前1个项目的最大收益为13，即 $F_1(3)=f_1(3)=13$
- 分配4万元给前1个项目的最大收益为14，即 $F_1(4)=f_1(4)=14$
- 分配5万元给前1个项目的最大收益为15，即 $F_1(5)=f_1(5)=15$

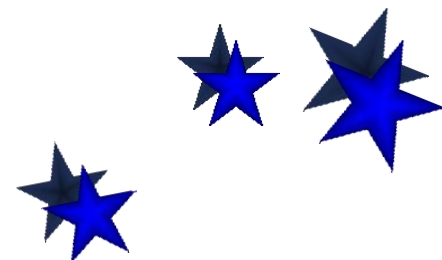
$$F_1=\{0,11,12,13,14,15\}$$



3.3 动态规划算法的典型应用

(2) 第2阶段：分别计算将 $x(0 \leq x \leq 5)$ 万元分配给前2个项目的最大收益

- 分配0万元给前2个项目的分配方案为：2项目0万元，前1个项目0万元，则最大收益 $F_2(0) = \max\{f_2(0), F_1(0)\} = \{0, 0\} = 0$
- 分配1万元给前2个项目的分配方案为：
 - ✓ 2项目0万元，前1个项目1万元，收益为： $f_2(0) + F_1(1) = 11$
 - ✓ 2项目1万元，前1个项目0万元，收益为： $f_2(1) + F_1(0) = 0$
 - ✓ 则最大收益 $F_2(1) = \max\{11, 0\} = 11$

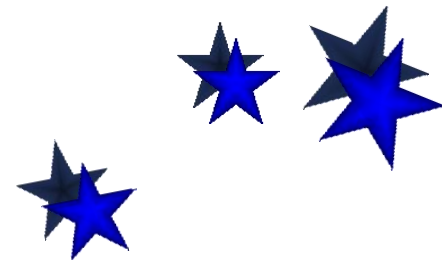


3.3 动态规划算法的典型应用

- 分配2万元给前2个项目的分配方案为：
 - ✓ 2项目0万元，前1个项目2万元，收益为： $f_2(0)+F_1(2)=12$
 - ✓ 2项目1万元，前1个项目1万元，收益为： $f_2(1)+F_1(1)=11$
 - ✓ 2项目2万元，前1个项目0万元，收益为： $f_2(2)+F_1(0)=2$
 - ✓ 则最大收益 $F_2(2)=\max\{12,11,2\}=12$
- 分配3万元给前2个项目的分配方案为：
 - ✓ 2项目0万元，前1个项目3万元，收益为： $f_2(0)+F_1(3)=13$
 - ✓ 2项目1万元，前1个项目2万元，收益为： $f_2(1)+F_1(2)=12$
 - ✓ 2项目2万元，前1个项目1万元，收益为： $f_2(2)+F_1(1)=16$
 - ✓ 2项目3万元，前1个项目0万元，收益为： $f_2(3)+F_1(0)=10$
 - ✓ 则最大收益 $F_2(3)=\max\{13,12,16,10\}=16$

3.3 动态规划算法的典型应用

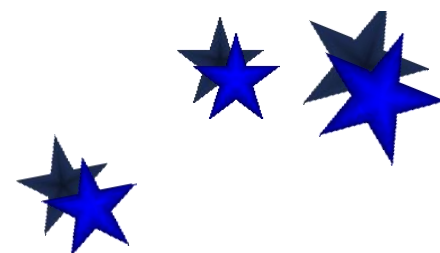
- 分配4万元给前2个项目的分配方案为：
 - ✓ 2项目0万元，前1个项目4万元，收益为： $f_2(0)+F_1(4)=14$
 - ✓ 2项目1万元，前1个项目3万元，收益为： $f_2(1)+F_1(3)=13$
 - ✓ 2项目2万元，前1个项目2万元，收益为： $f_2(2)+F_1(2)=17$
 - ✓ 2项目3万元，前1个项目1万元，收益为： $f_2(3)+F_1(1)=21$
 - ✓ 2项目4万元，前1个项目0万元，收益为： $f_2(4)+F_1(0)=15$
 - ✓ 则最大收益 $F_2(4)=\max\{14,13,17,21,15\}=21$



3.3 动态规划算法的典型应用

- 分配5万元给前2个项目的分配方案为：
 - ✓ 2项目0万元，前1个项目5万元，收益为： $f_2(0)+F_1(5)=15$
 - ✓ 2项目1万元，前1个项目4万元，收益为： $f_2(1)+F_1(4)=14$
 - ✓ 2项目2万元，前1个项目3万元，收益为： $f_2(2)+F_1(3)=18$
 - ✓ 2项目3万元，前1个项目2万元，收益为： $f_2(3)+F_1(2)=22$
 - ✓ 2项目4万元，前1个项目1万元，收益为： $f_2(4)+F_1(1)=26$
 - ✓ 2项目5万元，前1个项目0万元，收益为： $f_2(5)+F_1(0)=20$
 - ✓ 则最大收益 $F_2(4)=\max\{15,14,18,22,26,20\}=26$

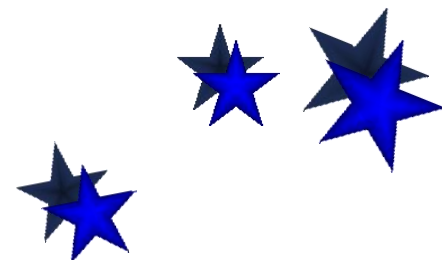
$$F_2=\{0,11,12,16,21,26\}$$



3.3 动态规划算法的典型应用

(3) 第3阶段：分别计算将 $x(0 \leq x \leq 5)$ 万元分配给前3个项目的最大收益

- 分配0万元给前3个项目的分配方案为：3项目0万元，前2个项目0万元，则最大收益 $F_3(0) = \max\{f_3(0), F_2(0)\} = \{0, 0\} = 0$
- 分配1万元给前3个项目的分配方案为：
 - ✓ 3项目0万元，前2个项目1万元，收益为： $f_3(0) + F_2(1) = 11$
 - ✓ 3项目1万元，前2个项目0万元，收益为： $f_3(1) + F_2(0) = 2$
 - ✓ 则最大收益 $F_3(1) = \max\{11, 2\} = 11$

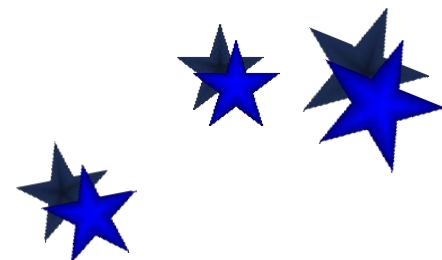


3.3 动态规划算法的典型应用

- 分配2万元给前3个项目的分配方案为：
 - ✓ 3项目0万元，前2个项目2万元，收益为： $f_3(0)+F_2(2)=12$
 - ✓ 3项目1万元，前2个项目1万元，收益为： $f_3(1)+F_2(1)=13$
 - ✓ 3项目2万元，前2个项目0万元，收益为： $f_3(2)+F_2(0)=10$
 - ✓ 则最大收益 $F_3(2)=\max\{12,13,10\}=13$
- 分配3万元给前3个项目的分配方案为：
 - ✓ 3项目0万元，前2个项目3万元，收益为： $f_3(0)+F_2(3)=16$
 - ✓ 3项目1万元，前2个项目2万元，收益为： $f_3(1)+F_2(2)=14$
 - ✓ 3项目2万元，前2个项目1万元，收益为： $f_3(2)+F_2(1)=21$
 - ✓ 3项目3万元，前2个项目0万元，收益为： $f_3(3)+F_2(0)=30$
 - ✓ 则最大收益 $F_3(3)=\max\{16,14,21,30\}=30$

3.3 动态规划算法的典型应用

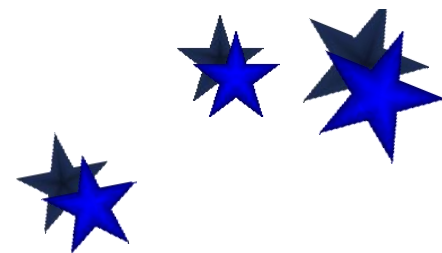
- 分配4万元给前3个项目的分配方案为：
 - ✓ 3项目0万元，前2个项目4万元，收益为： $f_3(0)+F_2(4)=21$
 - ✓ 3项目1万元，前2个项目3万元，收益为： $f_3(1)+F_2(3)=18$
 - ✓ 3项目2万元，前2个项目2万元，收益为： $f_3(2)+F_2(2)=22$
 - ✓ 3项目3万元，前2个项目1万元，收益为： $f_3(3)+F_2(1)=41$
 - ✓ 3项目4万元，前2个项目0万元，收益为： $f_3(4)+F_2(0)=32$
 - ✓ 则最大收益 $F_3(4)=\max\{21,18,22,41,32\}=41$



3.3 动态规划算法的典型应用

- 分配5万元给前3个项目的分配方案为：
 - ✓ 3项目0万元，前2个项目5万元，收益为： $f_3(0)+F_2(5)=26$
 - ✓ 3项目1万元，前2个项目4万元，收益为： $f_3(1)+F_2(4)=23$
 - ✓ 3项目2万元，前2个项目3万元，收益为： $f_3(2)+F_2(3)=26$
 - ✓ 3项目3万元，前2个项目2万元，收益为： $f_3(3)+F_2(2)=42$
 - ✓ 3项目4万元，前2个项目1万元，收益为： $f_3(4)+F_2(1)=43$
 - ✓ 3项目5万元，前2个项目0万元，收益为： $f_3(5)+F_2(0)=40$
 - ✓ 则最大收益 $F_3(5)=\max\{26,23,26,42,43,40\}=43$

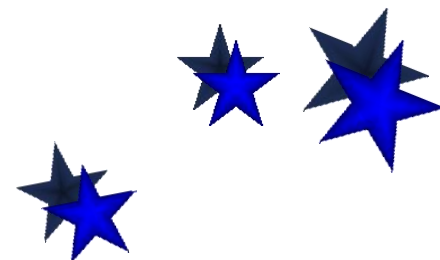
$$F_3=\{0,11,13,30,41,43\}$$



3.3 动态规划算法的典型应用

(4) 第4阶段：分别计算将 $x(0 \leq x \leq 5)$ 万元分配给前4个项目的最大收益

- 分配0万元给前4个项目的分配方案为：4项目0万元，前3个项目0万元，则最大收益 $F_4(0) = \max\{f_4(0), F_3(0)\} = \{0, 0\} = 0$
- 分配1万元给前4个项目的分配方案为：
 - ✓ 4项目0万元，前3个项目1万元，收益为： $f_4(0) + F_3(1) = 11$
 - ✓ 4项目1万元，前3个项目0万元，收益为： $f_4(1) + F_3(0) = 20$
 - ✓ 则最大收益 $F_4(1) = \max\{11, 20\} = 20$

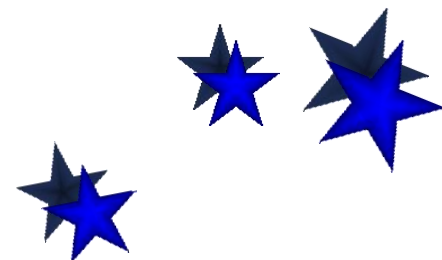


3.3 动态规划算法的典型应用

- 分配2万元给前4个项目的分配方案为：
 - ✓ 4项目0万元，前3个项目2万元，收益为： $f_4(0)+F_3(2)=13$
 - ✓ 4项目1万元，前3个项目1万元，收益为： $f_4(1)+F_3(1)=31$
 - ✓ 4项目2万元，前3个项目0万元，收益为： $f_4(2)+F_3(0)=21$
 - ✓ 则最大收益 $F_4(2)=\max\{13,31,21\}=31$
- 分配3万元给前4个项目的分配方案为：
 - ✓ 4项目0万元，前3个项目3万元，收益为： $f_4(0)+F_3(3)=30$
 - ✓ 4项目1万元，前3个项目2万元，收益为： $f_4(1)+F_3(2)=33$
 - ✓ 4项目2万元，前3个项目1万元，收益为： $f_4(2)+F_3(1)=32$
 - ✓ 4项目3万元，前3个项目0万元，收益为： $f_4(3)+F_3(0)=22$
 - ✓ 则最大收益 $F_4(3)=\max\{30,33,32,22\}=33$

3.3 动态规划算法的典型应用

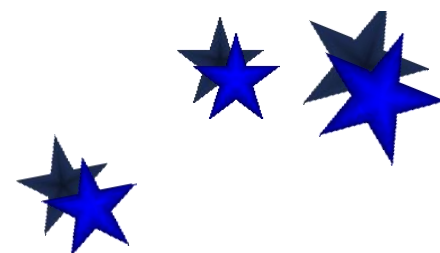
- 分配4万元给前4个项目的分配方案为：
 - ✓ 4项目0万元，前3个项目4万元，收益为： $f_4(0)+F_3(4)=41$
 - ✓ 4项目1万元，前3个项目3万元，收益为： $f_4(1)+F_3(3)=50$
 - ✓ 4项目2万元，前3个项目2万元，收益为： $f_4(2)+F_3(2)=34$
 - ✓ 4项目3万元，前3个项目1万元，收益为： $f_4(3)+F_3(1)=33$
 - ✓ 4项目4万元，前3个项目0万元，收益为： $f_4(4)+F_3(0)=23$
 - ✓ 则最大收益 $F_4(4)=\max\{41,50,34,33,23\}=50$



3.3 动态规划算法的典型应用

- 分配5万元给前4个项目的分配方案为：
 - ✓ 4项目0万元，前3个项目5万元，收益为： $f_4(0)+F_3(5)=43$
 - ✓ 4项目1万元，前3个项目4万元，收益为： $f_4(1)+F_3(4)=61$
 - ✓ 4项目2万元，前3个项目3万元，收益为： $f_4(2)+F_3(3)=51$
 - ✓ 4项目3万元，前3个项目2万元，收益为： $f_4(3)+F_3(2)=35$
 - ✓ 4项目4万元，前3个项目1万元，收益为： $f_4(4)+F_3(1)=34$
 - ✓ 4项目5万元，前3个项目0万元，收益为： $f_4(5)+F_3(0)=24$
 - ✓ 则最大收益 $F_4(5)=\max\{43,61,51,35,34,24\}=61$

$$F_4=\{0,20,31,33,50,61\}$$



3.3 动态规划算法的典型应用

➤ 解的追踪

(1) $F_4(5)$ 即为：前4个项目投资5万元的最大收益为61，即整个问题的解。

(2) 该解是给项目4投资了1万元，则前3个项目的投资为4万元。

(3) 在 $F_3(4)$ 的最大收益为41，该解是给项目3投资了3万元，则前2个项目的投资为1万元。

(4) 在 $F_2(1)$ 的最大收益为11，该解是给项目2投资了0万元，则前1个项目的投资为1万元。

(5) 在 $F_1(1)$ 的最大收益为11，该解是给项目1投资了1万元。



3.3 动态规划算法的典型应用

➤ 数据结构的设计

(1) 效益数组 f ，该数组的内容是逐行使用，在计算前 k 个项目时，只用 f_k 。因此可以使用一维数组实现。

(2) 最大收益数组 F ，在计算 F_k 时，仅反复使用 F_{k-1} ，因此可以使用一维数组 bp 存储 F_{k-1} ，计算 F_k 过程中使用一维数组 $temp$ 暂存， k 阶段决策结束后，将 $temp$ 数组的内容送入 bp 。最后一阶段结束后， $bp[5]$ 中的值就是最优解。

(3) 标记函数——追踪解：资源分配数组 $bonus$ ， $bonus[i,j]$ 记录 i 阶段投资 j 万元获得最大收益时，给项目 i 的投资金额。



3.3 动态规划算法的典型应用

➤ 根据数组内容给出决策结果

<i>bp</i>	0	1	2	3	4	5
	0	20	31	33	50	61

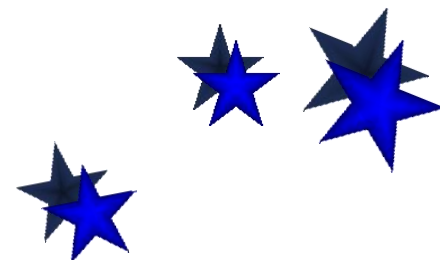
<i>bonus</i>		0	1	2	3	4	5
	1	0	1	2	3	4	5
	2	0	0	0	2	3	4
	3	0	0	1	3	3	4
	4	0	1	1	1	1	1



3.3 动态规划算法的典型应用

➤ 子问题界定和计算顺序

- 子问题界定：由参数 k 和 x 界定
 - (1) k : 考虑对前 $1, 2, \dots, k$ 个项目的投资——阶段
 - (2) x : 投资总钱数为 $x(0 \leq x \leq m)$
- 原始输入： $k = n, x = m$
 - (1) 子问题计算顺序： $k = 1, 2, \dots, n$
 - (2) 对于给定的 $k, x = 1, 2, \dots, m$



3.3 动态规划算法的典型应用

➤ 优化函数的递推方程

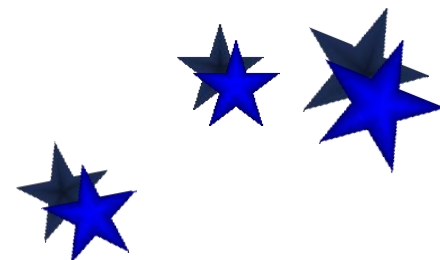
$F_k(x)$: x 元钱投给前 k 个项目最大效益

多步判断: 若知道 p 元钱 ($p \leq x$) 投给前 $k-1$ 个项目的最大效益 $F_{k-1}(p)$, 确定 x 元钱投给前 k 个项目的方案

递推方程和边界条件

$$F_k(x) = \max_{0 \leq x_k \leq x} \{f_k(x_k) + F_{k-1}(x - x_k)\} \quad k > 1$$

$$F_1(x) = f_1(x)$$



3.3 动态规划算法的典型应用

➤ 时间复杂度分析

备忘录表中有 m 行 n 列, 共计 mn 项

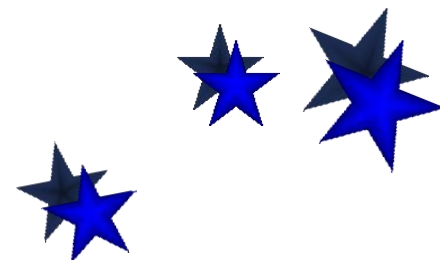
$$F_k(x) = \max_{0 \leq x_k \leq x} \{f_k(x_k) + F_{k-1}(x - x_k)\} \quad k > 1$$

$$F_1(x) = f_1(x)$$

x_k 有 $x+1$ 种可能的取值, 计算 $F_k(x)$ 项 ($2 \leq k \leq n, 1 \leq x \leq m$) 需要:

$x+1$ 次加法

x 次比较



3.3 动态规划算法的典型应用

对备忘录中所有的项求和：

$$\text{加法次数} \sum_{k=2}^n \sum_{x=1}^m (x+1) = \frac{1}{2}(n-1)m(m+3)$$

$$\text{比较次数} \sum_{k=2}^n \sum_{x=1}^m x = \frac{1}{2}(n-1)m(m+1)$$

$$W(n) = O(nm^2)$$

